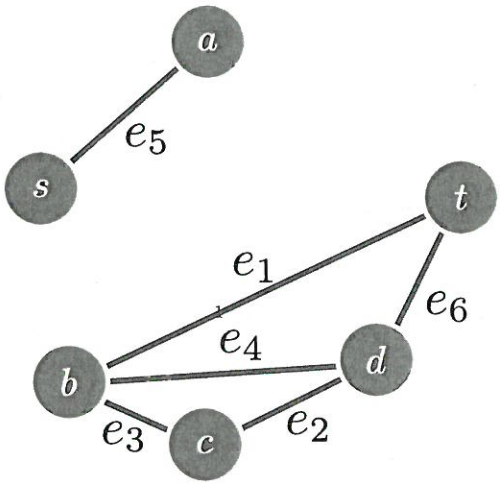

3. Show how to store the graph above as an incidence matrix ...

Answer Boundary Matrix
- columns labeled by edges
- rows labeled by verts
- (i,j) entry is $\begin{cases} 0, & \text{no edge} \\ 1, & \text{edge} \end{cases}$

Adjacency Matrix
- rows & cols are verts
- $(i,j) = 1 \iff (v_i, v_j) \in E$

4. ... and as an adjacency list.

Answer (why sparse matrix implementations are so small, compared to full matrix)

UF ds supports:
(1) union
(2) find
(0) initialize w/ a set.

Counting Connected Components

Algorithm 1 Count Components

Input: A graph $G = (V, E)$
Output: The number of connected components in G.

```
1: Initialize S, a union-find data structure with the vertex set V
2: E' ← E
3: while E' ≠ ∅ do
4:     e ← E'.pop()
5:     s1 ← S.find(e.head)
6:     s2 ← S.find(e.tail)
7:     if s1 ≠ s2 then
8:         S.union(s1, s2)
9:     end if
10: end while
11: return S.size()
```

1. In Algorithm 1, S is a data structure that keeps track of the connected components of a graph that is initially just the vertex set, and builds up to the full graph G by adding one vertex at a time. Without getting into more specifics, walk through running this algorithm for the example graph on the previous page. (We'll look into implementation details next, for now, just think 'what are the connected components' at each step as you are walking through).

   Answer

Aftr Ln 1:

$S = \{\{a\},\{b\},\{c\},\{d\},\{s\},\{t\}\}$ = {a, b, c, d, s, t}

$E' = \{\cancel{\emptyset}, e_2, e_3, e_4, e_5, e_6\}$ (s_1 = b, s_2 = t)

$e = e_1 = [b, t)$ (head, tail)

$s_1 \neq s_2 \Rightarrow$ union s_1, s_2

NOW:

$S = \{\{a\}, \{b, t,\}, \{c\}, \{d\}, \{s\}\}$ ($[b,t]$)

= {a, b—t, c, d, s}

Two Heuristics

Using the Fast union approach, we add two heuristics for improved runtime!

1. Depth: Associate a variable named 'rank' for each rooted tree. When unioning (merging) two rooted trees together, add the smaller to the larger (as measured by the rank variable). If there is a tie, pick arbitrarily and add one to the rank. Give this a try with the example graph.

   Answer

2. Fatten: When running a find operation (going up the parent until I find the root), directly connect each vertex encountered to the root. Give this a try with the example graph, using both the rank and fatten improvements.

   Answer

k op: $\Theta(k \alpha(n))$

↑ inverse ackermann fcn
= SUPER SLOW GROWING.